

Resumo P1 — Computação Paralela e Distribuída

Resumo para a primeira avaliação (Aulas 01 a 06)

Autor: Jeann Victor Batista

1 Classificação de Flynn

Classifica arquiteturas pelo número de fluxos de **instrução** e de **dados**. Não considera o tipo de memória (compartilhada $\times$ distribuída) — essa é uma subdivisão posterior, dentro do MIMD.

Sigla	Significado	Exemplo
SISD	Fluxo único de instrução, fluxo único de dados	Computador sequencial clássico; pipeline/superescalar de um único programa
SIMD	Fluxo único de instrução, múltiplos dados	GPU, ILLIAC IV, vetorização
MISD	Múltiplos fluxos de instrução, dado único	Sistemas redundantes (raro, ex.: votação para tolerância a falhas)
MIMD	Múltiplos fluxos de instrução, múltiplos dados	Clusters, multicomputadores, multiprocessadores

Atenção: pipeline/superescalar em UM programa sequencial continua SISD (paralelismo em nível de instrução, não fluxos independentes).

2 Redes de Interconexão (topologias)

Diâmetro (D): maior distância (em saltos) entre dois nós quaisquer. **Largura de bissecção (B):** nº mínimo de enlaces cuja remoção separa a rede em duas metades iguais. **Grau (g):** nº de conexões por nó. N = número total de nós.

Topologia	Diâmetro (D)	Bissecção (B)	Grau (g)	Nº de enlaces
Barramento	$N - 1$	1	1 (compartilhado)	$N - 1$
Anel (bidirecional)	$\lfloor N/2 \rfloor$	2	2	N
Malha 2D ($\sqrt{N} \times \sqrt{N}$)	$2(\sqrt{N} - 1)$	$\sqrt{N}$	2 a 4 (bordas menor)	$2N - 2\sqrt{N}$
Toro 2D ($\sqrt{N} \times \sqrt{N}$)	$2\lfloor \sqrt{N}/2 \rfloor$	$2\sqrt{N}$	4 (uniforme)	$2N$
Árvore / Fat tree	$2\log_2 N$	1 (raiz) / maior na fat tree	3 (interno)	$N - 1$
Hipercubo ($N = 2^n$)	$\log_2 N$	$N/2$	$\log_2 N$	$(N \log_2 N)/2$
Totalmente conectada	1	$\lfloor N^2/4 \rfloor$	$N - 1$	$N(N - 1)/2$

Toro = malha + *wraparound* (bordas conectadas): reduz o diâmetro pela metade em cada dimensão e uniformiza o grau. Fat tree: enlaces mais LARGOS perto da raiz (não mais estreitos) para aliviar o gargalo.

Redes dinâmicas (indiretas)

Rede	Custo (nº de pontos de chaveamento)	Bloqueante?
Crossbar $N \times N$	N^2 pontos de comutação — $O(N^2)$	Não bloqueante (qualquer permutação)
Multiestágio (Omega/Butterfly), switches 2×2	$\log_2 N$ estágios $\times (N/2)$ switches = $(N \log_2 N)/2$ — $O(N \log N)$	Bloqueante (algumas permutações geram conflito)

Redes estáticas/diretas (enlaces fixos) $\rightarrow$ típicas de memória distribuída. Redes dinâmicas/indiretas (switches) $\rightarrow$ típicas de memória compartilhada.

3 Granularidade e Paralelismo em Pipeline

Granularidade fina: tarefas pequenas, muita sincronização (boa quando a comunicação é barata). **Granularidade grossa:** tarefas grandes, pouca comunicação (boa quando a rede tem alta latência), mas sensível a desbalanceamento de carga. **Regra:** se o custo de comunicação por tarefa é aproximadamente constante, aumentar o tamanho da tarefa

eleva a razão computação/comunicação.

Pipeline com k estágios, tempo t por estágio, n itens

$$T_{\text{pipeline}} = k t + (n - 1) t = (k + n - 1) t$$

$$S_{\text{pipeline}} = \frac{n k}{k + n - 1}$$

Quando $n \rightarrow \infty$, o speedup do pipeline tende ao número de estágios k (nunca ultrapassa k). Se um estágio fica mais lento (gargalo), a vazão em regime passa a ser limitada por esse estágio (1 item a cada t_{gargalo} segundos).

4 Métricas de Desempenho

Speedup e Eficiência

$$S(p) = \frac{T_1}{T_p}$$

$$E(p) = \frac{S(p)}{p}$$

T_1 = tempo do melhor algoritmo sequencial; T_p = tempo com p processadores. $E(p) = 1 \rightarrow$ uso perfeito; $E(p)$ baixo $\rightarrow$ muitos recursos ociosos/overhead.

Lei de Amdahl (problema de tamanho FIXO — escalabilidade forte)

$$S(p) = \frac{1}{f + \frac{1-f}{p}}$$

f = fração estritamente sequencial do programa. Quando $p \rightarrow \infty$, $S(p) \rightarrow 1/f$ (teto teórico do speedup). Ex.: $f = 0,1 \Rightarrow$ speedup máximo = 10, mesmo com infinitos processadores.

Lei de Gustafson-Barsis (problema ESCALADO — escalabilidade fraca)

$$S(p) = p - f(p - 1) = f + (1 - f)p$$

f = fração serial medida NA EXECUÇÃO PARALELA (do problema já escalado, não do problema sequencial). Aqui não há termo dividido por p — o speedup escalado cresce aproximadamente de forma linear com p , diferente de Amdahl. Amdahl e Gustafson não se contradizem: partem de premissas diferentes (problema fixo $\times$ problema escalado).

Métrica de Karp-Flatt (fração serial experimental)

$$e = \frac{\frac{1}{S(p)} - \frac{1}{p}}{1 - \frac{1}{p}}$$

Calculada a partir de speedups MEDIDOS (não assumidos). Se e cresce com $p \rightarrow$ indica overhead de paralelização crescente (comunicação/sincronização), não apenas fração serial fixa. Se e fica praticamente constante $\rightarrow$ a perda vem mesmo da fração serial do programa.

Redundância, Qualidade e Utilização

$$R = \frac{\text{operações realizadas (paralelo)}}{\text{operações mínimas necessárias}}$$

$$Q = \frac{S(p)}{R} \quad (\text{Qualidade} = \text{Speedup} / \text{Redundância})$$

$$U = \frac{T_{\text{compute total}}}{N \times T_{\text{paralelo}}} \quad (\text{Utilização})$$

$R \geq 1$ sempre (trabalho extra por duplicação/overhead). Q próximo de 1 $\rightarrow$ bom aproveitamento; $Q < 1 \rightarrow$ perdas por redundância/overhead. $U \rightarrow$ fração do tempo dos N processadores realmente em computação útil ($1 - U =$ fração perdida em ociosidade, espera ou comunicação).

Escalabilidade forte $\times$ fraca — resumo

	Escalabilidade Forte	Escalabilidade Fraca
Tamanho do problema	Fixo	Cresce junto com p
Lei associada	Amdahl	Gustafson-Barsis
Ideal	Tempo cai proporcionalmente a p	Tempo permanece aproximadamente constante quando n e p crescem juntos

5 Fundamentos de Sistemas Distribuídos

Definição de Tanenbaum: sistema visto pelo usuário como um único sistema coerente. **Definição de Lamport:** “um sistema distribuído é aquele em que a falha de um computador que você nem sabia que existia pode inutilizar o seu” — evidencia as falhas parciais.

Middleware: camada que oculta a heterogeneidade das plataformas e oferece interfaces comuns às aplicações.

Transparência: mascarar do usuário aspectos da distribuição — tem custo de desempenho, não é sempre desejável maximizá-la (ex.: distinguir recurso morto de recurso lento é difícil; *timeout* sozinho não garante certeza).

Política $\times$ Mecanismo: mecanismo = como fazer (ex.: lista de permissões); política = regra de decisão (ex.: quem pode gravar). Separar os dois favorece extensibilidade.

Abertura/Interoperabilidade $\times$ Portabilidade: componentes de fornecedores diferentes que seguem a mesma especificação e cooperam = interoperabilidade. Executar sem modificação em outro sistema com as mesmas interfaces = portabilidade. Sistemas abertos favorecem componentes pequenos e bem definidos (não grandes e acoplados).

Escalabilidade

Vertical (*scale-up*): aumentar a capacidade de uma máquina — esbarra em limites físicos/econômicos, mantém ponto único de falha. **Horizontal (*scale-out*):** adicionar mais máquinas — exige particionamento, replicação e consistência. Técnicas de escalabilidade de comunicação: DNS *round-robin*, balanceadores de carga (HAProxy, NGINX). Replicação melhora leituras, mas cada escrita precisa ser propagada a todas as réplicas (custo de coordenação).

As 8 Falácias da Computação Distribuída (Peter Deutsch)

- | | |
|----------------------------------|---------------------------------|
| 1. A rede é confiável | 5. A topologia não muda |
| 2. A latência é zero | 6. Há apenas um administrador |
| 3. A largura de banda é infinita | 7. O custo de transporte é zero |
| 4. A rede é segura | 8. A rede é homogênea |

Cada falácia é independente: resolver uma (ex.: TLS resolve “a rede é segura”) não resolve as demais.

Transações ACID

Atomicidade	tudo ou nada — se falha no meio, desfaz o que já foi feito
Consistência	a transação não viola os invariantes/regras do sistema
Isolamento (<i>Isolation</i>)	transações concorrentes não interferem entre si
Durabilidade	após confirmada, a alteração persiste mesmo com queda de energia

6 Protocolo Chord (DHT — *Distributed Hash Table*)

Identificadores de m bits formam um anel lógico de 0 a $2^m - 1$ (nós e chaves recebem ID por hash, ex.: SHA-1).

$$\text{sucessor(chave)} = \text{primeiro nó no anel com ID} \geq \text{hash(chave)}$$

Circular: dá a volta no anel se necessário.

$$\text{finger}_n[i] = \text{sucessor}(n + 2^i), \quad i = 0, \dots, m - 1$$

$$\text{Custo de busca: } O(N) \text{ (sem finger table)} \rightarrow O(\log N) \text{ (com finger table)}$$

Se sai um nó, suas chaves passam para o SUCESSOR (não o predecessor). A estabilização periódica atualiza ponteiros de sucessor/predecessor e as finger tables. Redes P2P não estruturadas (sem DHT) NÃO garantem $O(\log N)$ — dependem de inundação/busca aleatória.

7 Estilos Arquiteturais (Aula 06) — resumo rápido

Estilo	Acoplamento / comunicação	Ponto-chave para provas
Cliente-Servidor	Síncrono (bloqueante); um pode ser cliente de outro servidor	Simplicidade × ponto único de falha/gargalo
N-Tier (camadas)	Hierárquico (apresentação → lógica → dados)	Mais camadas = mais latência, não menos
SOA	Serviços grandes, governança centralizada, Service Registry	Acoplamento fraco entre serviços, banco geralmente compartilhado
Microserviços	Serviços pequenos, banco independente por serviço, síncrono ou assíncrono	Escalabilidade granular (não precisa replicar tudo)
Event-Driven	Assíncrono (produção → publicação → consumo)	Produtor não conhece consumidor; consistência eventual; mais difícil rastrear
Peer-to-Peer (P2P)	Todo nó é cliente e servidor; rede <i>overlay</i> sobre a física	Sem servidor central; <i>overlay</i> ≠ topologia física (vizinhos no overlay podem estar longe fisicamente)
Baseada em DHT	Estrutura de hash distribuída (ex.: Chord)	Busca eficiente $O(\log N)$; ver seção 6

Componentes fortemente acoplados (*tightly coupled*) = forte dependência entre si; fracamente acoplados (*loosely coupled*) = pouca dependência (ex.: comunicação via filas/eventos assíncronos REDUZ o acoplamento, não aumenta).

Resumo compilado a partir dos slides das Aulas 01–06 e da Lista de Exercícios (Nível Avançado), DCE540 — UNIFAL-MG.